UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería — Curso: Servicios Cloud

DISEÑO E IMPLEMENTACIÓN DE INFRAESTRUCTURA COMO CÓDIGO PARA LA GESTIÓN MULTI-CLOUD DE 50 SUCURSALES MEDIANTE TERRAFORM Y PRUEBAS LOCALES

Docente	Eleodoro Orbegoso Carrera
Integrantes	Icochea Rodriguez, Thiago Paolo Gonzales Aguilar, Carlos Enrique Giussepe Huamani Pereira, Eddyson Cesar Remuzgo Tovar, Huber Eduardo Quispe Saavedra, Karen Meylin

Lima, 16 de octubre de 2026

Año académico 2026

ÍNDICE

ÍNDICE..... 2
RESUMEN..... 6
1. INTRODUCCIÓN..... 8
2. PLANTEAMIENTO DEL PROBLEMA..... 8
Contexto empresarial..... 8
Problematica..... 8
Objetivo general..... 9
Objetivos especificos..... 9
Requisitos funcionales (RF)..... 9
Requisitos no funcionales (RNF)..... 10
Restricciones tecnicas..... 10
Riesgos..... 11
Supuestos..... 11
Criterios de aceptacion..... 11
3. OBJETIVOS..... 12
Objetivo general..... 12
Objetivos específicos..... 12
4. MARCO TEÓRICO..... 12
4.1 Infraestructura como Código..... 12
4.2 Terraform..... 13
4.3 AWS..... 13
4.4 Google Cloud Platform..... 13
4.5 CI/CD..... 13
4.6 Terraform State..... 13
4.7 Infrastructure Drift..... 14

4.8 Seguridad Cloud....14
5. ANÁLISIS DE REQUISITOS....14
6. ARQUITECTURA PROPUESTA....17
1. Patron de diseno....17
2. Interfaz comun entre proveedores....17
3. Extensibilidad (RNF)....18
4. Supuesto academico de distribucion AWS/GCP....18
5. Entornos y estado....18
6. Limitacion de diseno documentada: regiones AWS....19
Recursos por sucursal....19
Justificacion de herramientas....20
Recursos por sucursal....21
Diferencias tecnicas respecto a AWS (documentadas, no forzadas a una abstraccion comun)....22
Justificacion de herramientas....22
7. IMPLEMENTACIÓN....23
7.1 Estructura del repositorio....23
7.2 Módulos Terraform....24
7.3 Configuración de sucursales....24
7.4 Redes, servidores, seguridad y monitoreo....24
8. GESTIÓN DEL ESTADO....25
Diseno empresarial (AWS)....25
Diseno empresarial (GCP)....26
Laboratorio local (lo unico que se ejecuta realmente)....26
Riesgos de almacenar informacion sensible en el state....27
9. PIPELINE CI/CD....28
Flujo comun AWS + GCP....28
Procedimiento de aprobacion....28

Por que el pipeline NUNCA ejecuta `terraform apply` contra AWS/GCP....28
Habilitar el despliegue real en el futuro....29
10. SEGURIDAD....30
Lo implementado en este repositorio....30
Uso futuro (empresarial, no implementado por falta de credenciales)....30
Que NO existe en este repositorio (verificado)....31
11. INFRASTRUCTURE DRIFT....31
Por que un recurso local y no uno de AWS/GCP....32
Diferencias entre Drift local y Drift en un proveedor cloud real....32
Ejecucion....32
Importante....33
12. PRUEBAS LOCALES....34
CP01 — Validacion sintactica de Terraform....35
CP02 — Validacion de modulos AWS....35
CP03 — Validacion de modulos GCP....36
CP04 — Configuracion de 50 sucursales....36
CP05 — Dos servidores por sucursal....36
CP06 — Redes independientes....37
CP07 — Reglas de seguridad....37
CP08 — Configuracion de monitoreo....37
CP09 — Pruebas con mock_provider....37
CP10 — Ejecucion de GitHub Actions....38
CP11 — Verificacion de secretos....38
CP12 — Consulta de Terraform State....39
CP13 — Deteccion de Infrastructure Drift....39
CP14 — Reconciliacion de Drift....39
CP15 — Incorporacion de una nueva sucursal....39
Laboratorio Docker — ejecucion real (GitHub Actions)....40

CP16 — Limpieza del laboratorio....41
13. RESULTADOS....43
Limitaciones....44
14. DISCUSIÓN....44
15. CONCLUSIONES....45
16. RECOMENDACIONES....46
17. REPARTICIÓN DE TAREAS....46
Resumen por integrante....46
Detalle de tareas....47
Icochea Rodriguez, Thiago Paolo — Coordinador y arquitecto....47
Gonzales Aguilar, Carlos Enrique Giussepe — Infraestructura AWS....48
Huamani Pereira, Eddyson Cesar — Infraestructura GCP....48
Remuzgo Tovar, Huber Eduardo — CI/CD y seguridad....48
Quispe Saavedra, Karen Meylin — State, Drift y pruebas....49
Tareas compartidas....49
18. REFERENCIAS BIBLIOGRÁFICAS....50
19. ANEXOS....51
Anexo A — Comandos principales....51
Anexo B — Enlace al repositorio GitHub....51
Anexo C — Guía de ejecución local completa....51

RESUMEN

Este informe documenta el diseño, la implementación y la validación local de una solución de Infraestructura como Código (IaC) con Terraform para administrar **50 sucursales** de una empresa mediante un patrón común, sobre **AWS y Google Cloud Platform**. Debido a que el proyecto **no cuenta con credenciales de AWS ni de GCP**, la solución se valida íntegramente mediante `terraform validate`, `terraform test` con `mock_provider`, un script de validación de esquema en Python y dos laboratorios con recursos 100% locales (`hashicorp/local`): uno para el ciclo de vida del Terraform State y otro para la detección y reconciliación de Infrastructure Drift. Se implementaron 8 módulos Terraform reutilizables (red, cómputo, seguridad y monitoreo para cada proveedor), orquestados por un módulo compuesto (`branch-stack`) que procesa las 50 sucursales a partir de un único archivo de datos (`branches.yaml`), sin duplicar código. Se diseñó un pipeline CI/CD común a ambos proveedores (GitHub Actions) con controles de formato, validación, pruebas, análisis estático de seguridad (TFLint, Checkov) y escaneo de secretos (gitleaks), y un procedimiento de aprobación mediante Pull Request. Las 27 pruebas automatizadas de Terraform (`mock_provider`) pasaron exitosamente, así como la validación de escalabilidad de las 50 sucursales y los dos laboratorios locales (State y Drift). El laboratorio Docker de 2 sucursales (4 contenedores, HTTP, conectividad interna y aislamiento de red) y el monitoreo local se ejecutaron realmente en GitHub Actions —no en el equipo de desarrollo local, que no tiene Docker Desktop operativo— con resultado exitoso y evidencia real capturada. El informe distingue en todo momento lo que se ejecutó realmente de lo que queda documentado como diseño para un despliegue empresarial futuro con credenciales reales.

Palabras clave: Infraestructura como Código, Terraform, AWS, Google

Cloud Platform, CI/CD, Infrastructure Drift, Terraform State, DevOps.

1. INTRODUCCIÓN

La Infraestructura como Código (IaC) permite definir, versionar y automatizar el aprovisionamiento de recursos cloud mediante archivos de configuración declarativos, en lugar de procedimientos manuales. Herramientas como Terraform de HashiCorp permiten administrar múltiples proveedores cloud (AWS, GCP, Azure, entre otros) con una sintaxis común (HCL), manteniendo un estado que refleja la infraestructura real administrada.

En escenarios empresariales con decenas de sucursales, sedes o unidades de negocio que requieren una infraestructura tecnológica similar, la IaC resuelve un problema central: aplicar el mismo patrón de red, cómputo, seguridad y monitoreo de forma consistente, auditable y escalable, sin copiar manualmente la configuración sucursal por sucursal. Este trabajo aborda ese problema para el caso académico "cincuenta sucursales con un patrón común", bajo la restricción adicional de no contar con credenciales reales de AWS ni GCP, lo que exige una estrategia de validación basada en simulación, pruebas locales y proveedores simulados (`mock_provider`).

2. PLANTEAMIENTO DEL PROBLEMA

Contexto empresarial

Una empresa con 50 sucursales necesita administrar su infraestructura tecnologica mediante un patron comun de aprovisionamiento, usando AWS y Google Cloud Platform, con un proceso de revision/aprobacion comun a ambos proveedores y sin almacenar secretos en el repositorio.

Problematica

Aprovisionar y mantener manualmente la infraestructura de 50 sucursales (o copiando configuracion sucursal por sucursal) es propenso a errores, dificil de auditar, dificil de escalar y no detecta cuando alguien modifica

manualmente un recurso en produccion (Infrastructure Drift).

Objetivo general

Disenar, implementar y validar localmente una solucion de Infraestructura como Codigo (IaC) con Terraform que aprovisione el patron comun de 50 sucursales en AWS y GCP, de forma reutilizable, segura, probada y documentada, sin requerir credenciales cloud para su demostracion.

Objetivos especificos

1. Disenar una arquitectura multi-cloud con interfaz de configuracion comun entre AWS y GCP.
2. Implementar modulos Terraform reutilizables (red, computo, seguridad, monitoreo) para ambos proveedores.
3. Demostrar que la solucion escala a 50 sucursales sin duplicar codigo.
4. Implementar una estrategia de pruebas sin credenciales cloud (`terraform test` + `mock_provider`, pruebas de esquema en Python, laboratorio Docker).
5. Disenar la gestion del Terraform State (estrategia empresarial + laboratorio local funcional).
6. Implementar un pipeline CI/CD comun a ambos proveedores con controles de seguridad (TFLint, Checkov, gitleaks) y un procedimiento de aprobacion.
7. Demostrar, con un recurso local real, el ciclo completo de deteccion y reconciliacion de Infrastructure Drift.
8. Documentar el proyecto en formato academico IEEE y publicarlo en un repositorio GitHub.

Requisitos funcionales (RF)

ID	Requisito
RF-01	Cada sucursal debe tener una red independiente/segmentada

RF-02	Cada sucursal debe tener dos servidores
RF-03	Cada sucursal debe tener reglas de seguridad
RF-04	Cada sucursal debe tener monitoreo
RF-05	La solucion debe soportar AWS y GCP
RF-06	Debe ser posible agregar una sucursal nueva sin duplicar modulos
RF-07	Debe ser posible modificar una sucursal sin afectar a las demas
RF-08	Los entornos deben estar separados (development/staging/production)
RF-09	El estado de Terraform debe estar aislado por entorno

Requisitos no funcionales (RNF)

ID	Requisito
RNF-01	Extensibilidad: agregar una sucursal es un cambio de datos, no de codigo
RNF-02	Reproducibilidad: cualquier persona con el repo puede ejecutar las validaciones locales sin credenciales
RNF-03	Seguridad: ningun secreto en el repositorio; principio de minimo privilegio documentado
RNF-04	Eficiencia de recursos: el laboratorio local debe funcionar en un equipo de 8 GB RAM sin GPU
RNF-05	Mantenibilidad: modulos documentados (README, variables y outputs descritos)

Restricciones tecnicas

- Sin credenciales de AWS ni GCP (no se ejecuta `terraform apply` real).

- Equipo de desarrollo: Windows 11, 8 GB RAM, sin GPU dedicada.
- Docker Desktop no operativo en el equipo de desarrollo local (requiere reiniciar Windows para activar WSL2, reinicio que se decidio no realizar): el laboratorio Docker se ejecuta realmente en GitHub Actions (runner `ubuntu-latest`, con Docker preinstalado) en vez de en el equipo local (ver `lab/docker/README.md`).

Riesgos

Riesgo	Mitigacion
Confundir una validacion con mocks con un despliegue real	Documentacion explicita en cada README y en el plan de pruebas sobre que SI y que NO valida cada mecanismo
Drift no detectado en produccion	Workflow programado `drift-check.yml` (demostrado localmente)
Secretos filtrados accidentalmente	`.gitignore` + `gitleaks` + verificacion adicional en CI
Limitaciones de recursos del equipo de desarrollo	Laboratorio Docker reducido a 2 sucursales / 4 contenedores; monitoreo ligero en vez de Prometheus/Grafana

Supuestos

- La distribucion de sucursales entre AWS y GCP (25/25) es un supuesto academico para esta demostracion (seccion 5 de `docs/architecture/overview.md`).
- Los valores de `ami_id`/`image` de los modulos de computo son ejemplos academicos, no validados contra una cuenta real de AWS/GCP.
- `project_id` de GCP (`academic-demo-project`) es un identificador de ejemplo, no un proyecto GCP real.

Criterios de aceptacion

Ver `docs/academic/plan-de-pruebas.md` (CP01-CP16): el proyecto se considera conforme cuando todas las pruebas (CP01-CP16) pasan con evidencia real, sin

afirmarse como aprobadas pruebas que no se ejecutaron. Las pruebas que dependen de Docker (laboratorio de 2 sucursales, monitoreo local) se ejecutaron realmente en GitHub Actions en vez de en el equipo de desarrollo local (ver `docs/academic/plan-de-pruebas.md`, seccion "Laboratorio Docker — ejecucion real").

3. OBJETIVOS

Objetivo general

Diseñar, implementar y validar localmente una solución de Infraestructura como Código con Terraform que aprovisione el patrón común de 50 sucursales en AWS y GCP, de forma reutilizable, segura, probada y documentada, sin requerir credenciales cloud para su demostración.

Objetivos específicos

9. Diseñar una arquitectura multi-cloud con interfaz de configuración común entre AWS y GCP.
10. Implementar módulos Terraform reutilizables para ambos proveedores.
11. Demostrar que la solución escala a 50 sucursales sin duplicar código.
12. Implementar una estrategia de pruebas sin credenciales cloud.
13. Diseñar la gestión del Terraform State (estrategia empresarial + laboratorio local funcional).
14. Implementar un pipeline CI/CD común con controles de seguridad.
15. Demostrar el ciclo completo de detección y reconciliación de Infrastructure Drift.
16. Documentar el proyecto en formato académico IEEE y publicarlo en GitHub.

4. MARCO TEÓRICO

4.1 Infraestructura como Código

La Infraestructura como Código es la práctica de gestionar y aprovisionar

infraestructura computacional mediante archivos de definición legibles por máquina, en lugar de configuración manual o herramientas interactivas [1].

4.2 Terraform

Terraform es una herramienta de IaC de código abierto desarrollada por HashiCorp que usa el lenguaje declarativo HCL (HashiCorp Configuration Language) para definir recursos de múltiples proveedores cloud, manteniendo un archivo de estado que mapea la configuración declarada con los recursos reales [2]. Desde la versión 1.6, incorpora `terraform test`, un framework de pruebas nativo que soporta `mock_provider` para simular proveedores sin realizar llamadas reales a sus APIs [3].

4.3 AWS

Amazon Web Services (AWS) es la plataforma cloud de Amazon, que ofrece servicios de cómputo (EC2), redes (VPC), seguridad (Security Groups, IAM) y observabilidad (CloudWatch), entre otros [4].

4.4 Google Cloud Platform

Google Cloud Platform (GCP) es la plataforma cloud de Google, con servicios equivalentes: Compute Engine, VPC Network, Firewall Rules y Cloud Monitoring [5].

4.5 CI/CD

La integración y entrega continuas (CI/CD) automatizan la construcción, prueba y despliegue de software mediante pipelines declarativos. GitHub Actions es la plataforma de CI/CD nativa de GitHub, basada en workflows YAML disparados por eventos del repositorio [6].

4.6 Terraform State

El estado de Terraform es un archivo (por defecto JSON) que registra los

recursos que Terraform administra y sus atributos, permitiendo calcular diferencias entre la configuración declarada y la infraestructura real [7].

Para equipos, HashiCorp recomienda backends remotos con bloqueo (locking) para evitar condiciones de carrera entre ejecuciones concurrentes [8].

4.7 Infrastructure Drift

El "drift" de infraestructura ocurre cuando el estado real de un recurso diverge de lo declarado en el código, típicamente por cambios manuales realizados fuera de la herramienta de IaC. Terraform detecta el drift mediante `terraform plan`, que compara el estado almacenado (opcionalmente refrescado contra el proveedor real) con la configuración declarada [9].

4.8 Seguridad Cloud

Las prácticas de seguridad cloud relevantes para este proyecto incluyen el principio de mínimo privilegio en IAM, el cifrado de datos en reposo y en tránsito, la gestión de secretos fuera del control de versiones, y el análisis estático de configuraciones IaC (p. ej., con Checkov) para detectar configuraciones inseguras antes del despliegue [10].

5. ANÁLISIS DE REQUISITOS

Requisito	Solucion	Herramienta	Prueba	Evidencia
RF-01 Red independiente por sucursal	`modules/aws/network`, `modules/gcp/network`	Terraform, AWS VPC, GCP VPC Network	CP02, CP03, CP06	`terraform/tests/aws_network.tftest.hcl`, `gcp_network.tftest.hcl`
RF-02 Dos servidores por sucursal	`modules/aws/compute`, `modules/gcp/compute`	Terraform, EC2, Compute Engine	CP02, CP03, CP05	`aws_compute.tftest.hcl`, `gcp_compute.tftest.hcl`

RF-03 Reglas de seguridad	`modules/aws/security`, `modules/gcp/security`	Terraform, Security Groups, Firewall Rules	CP02, CP03, CP07	`aws_security.tftest.hcl`, `gcp_security.tftest.hcl`
RF-04 Monitoreo	`modules/aws/monitoring`, `modules/gcp/monitoring`	Terraform, CloudWatch, Cloud Monitoring	CP02, CP03, CP08	`aws_monitoring.tftest.hcl`, `gcp_monitoring.tftest.hcl`
RF-05 Soporte AWS + GCP	`modules/branch-stack` (filtra por `provider`)	Terraform	CP02, CP03, CP04	`branch_stack.tftest.hcl`
RF-06 Agregar sucursal sin duplicar modulos	`config/branches/branches.yaml` + `for_each`	Terraform `yamldecode`, Python	CP04, CP15	`validate_branches_schema.py`, `branch_stack.tftest.hcl`
RF-07 Modificar una sucursal sin afectar a las demas	`for_each` por `id` (clave estable) en todos los modulos	Terraform	CP04	`terraform plan` dirigido (`-target`) documentado en `docs/user-guide`
RF-08 Entornos separados	`terraform/environments/{development,staging,production}`	Terraform	CP01	`terraform validate` por entorno (ver `docs/evidence`)
RF-09 Estado aislado por entorno	`backend "local"` con `path` propio por entorno (diseno real: prefijo S3/GCS por entorno)	Terraform	CP12	`docs/architecture/terraform-state-strategy.md`

Sin credenciales cloud	Modulos sin `data` sources que requieran API real; valores de ejemplo	Terraform	CP01, CP02, CP03	`terraform validate` sin backend remoto
Sin secretos en el repositorio	`.gitignore`, `gitleaks`, verificacion adicional en CI	git, gitleaks, GitHub Actions	CP11	`.github/workflows/terraform-security.yml`
Deteccion de Infrastructure Drift	`lab/drift` (provider local)	Terraform, PowerShell	CP13, CP14	`lab/drift/run_drift_demo.ps1`, `docs/evidence/drift/`
Pipeline CI/CD comun AWS+GCP	`.github/workflows/terraform-ci.yml`	GitHub Actions	CP10	Runs reales (fmt, validate, test, TFLint, Checkov, gitleaks, drift-check) en https://github.com/ThiagoIcochea/ServiciosCloudIAC/actions
Laboratorio local de 2 sucursales	`lab/docker`	Docker Compose, GitHub Actions (runner ubuntu-latest)	CP16	Run real exitoso: https://github.com/ThiagoIcochea/ServiciosCloudIAC/actions/runs/37832592874, `docs/evidence/docker-lab/`

Esta matriz se actualiza al finalizar la implementacion (seccion 4 del

enunciado). Version vigente: ver fecha del ultimo commit que modifico este archivo.

6. ARQUITECTURA PROPUESTA

1. Patron de diseno

Cada sucursal es una instancia de un **patron comun** descrito como datos (`config/branches/branches.yaml`), no como codigo repetido. El patron exige, por sucursal:

17. Una red independiente (VPC/VPC Network + subnet dedicada).
18. Dos servidores (`web`, `app`).
19. Reglas de seguridad (Security Group en AWS / Firewall Rules en GCP).
20. Monitoreo (CloudWatch en AWS / Cloud Monitoring en GCP).

2. Interfaz comun entre proveedores

Para que agregar una sucursal sea un cambio de **datos** y no de **codigo**, los 8 modulos base comparten la misma forma de entrada logica aunque cada proveedor exponga argumentos distintos:

Concepto logico (comun)	**AWS (`modules/aws/*`)**	**GCP (`modules/gcp/*`)**
Red de la sucursal	`aws_vpc` + `aws_subnet`	`google_compute_network` + `google_compute_subnetwork`
Tamano de servidor (`size`)	`instance_type` (t3.micro/small/medium)	`machine_type` (e2-micro/small/medium)
Servidor	`aws_instance`	`google_compute_instance`
Regla de seguridad	`aws_security_group_rule`	`google_compute_firewall` (1 recurso por regla)
Monitoreo	`aws_cloudwatch_log_group` + `aws_cloudwatch_metric_alarm`	`google_monitoring_alert_policy`

Las diferencias tecnicas reales (GCP no tiene un objeto "Security Group"

contenedor; AWS fija la region a nivel de provider y GCP a nivel de recurso; GCP gestiona la retencion de logs a nivel de proyecto) se documentan en el README de cada modulo y se respetan explicitamente en vez de forzar una abstraccion artificial.

3. Extensibilidad (RNF)

Agregar la sucursal 51 es agregar una entrada a `branches.yaml` (o regenerar el archivo con `scripts/generate_branches.py` ajustando los parametros). Ningun archivo `.tf` cambia. Esto se verifica automaticamente en `terraform/tests/branch_stack.tftest.hcl` y en `scripts/validate_branches_schema.py`.

4. Supuesto academico de distribucion AWS/GCP

El enunciado exige distribuir las 50 sucursales entre AWS y GCP "para la demostracion" y pide indicar explicitamente que es un supuesto academico. Esta demo usa una distribucion **25 AWS / 25 GCP** (sucursales con id impar -> AWS, id par -> GCP), ver `scripts/generate_branches.py`. En un caso real la asignacion dependeria de criterios de negocio ajenos a este laboratorio (contratos marco, presencia regional de cada proveedor, costos negociados, latencia a los puntos de venta, etc.).

5. Entornos y estado

Entorno	Sucursales (de las 50)	Proposito
`production`	40 (ids 001-040)	Sucursales operativas
`staging`	6 (ids 041-046)	Validacion previa a produccion
`development`	4 (ids 047-050)	Pruebas de nuevas sucursales/modulos

Cada entorno tiene su propio estado de Terraform (aislamiento de blast radius): ver `docs/architecture/terraform-state-strategy.md`.

6. Limitacion de diseno documentada: regiones AWS

Todas las sucursales AWS de esta demo comparten una unica region (`sa-east-1`) porque el provider `aws` de Terraform fija la region a nivel de **provider**, no de recurso. Soportar varias regiones AWS en el mismo `apply` exige alias de provider (`aws.sa_east_1`, `aws.us_east_1`, ...) pasados explicitamente a cada modulo — una extension valida pero fuera del alcance de este laboratorio academico sin credenciales. GCP si combina varias regiones con un unico provider porque `region`/`zone` son argumentos de **recurso** (ver `modules/gcp/network` y `modules/gcp/compute`).

Figura 1. Arquitectura general - 50 sucursales multi-cloud

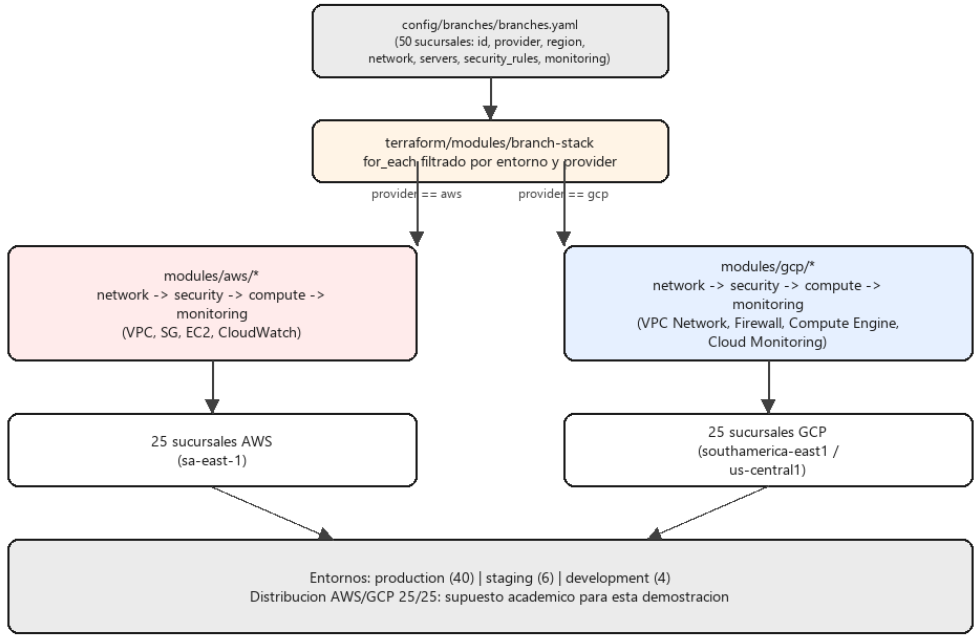


Figura 1. Arquitectura general de las 50 sucursales.

Recursos por sucursal

Recurso	Cantidad	Modulo
aws_vpc	1	aws/network
aws_subnet	1	aws/network
aws_internet_gateway	1	aws/network

aws_route_table (+ association)	1	aws/network
aws_security_group	1	aws/security
aws_security_group_rule	N (segun security_rules)	aws/security
aws_instance	2	aws/compute
aws_cloudwatch_log_group	1	aws/monitoring
aws_cloudwatch_metric_alarm	2 (una por servidor)	aws/monitoring

Para las 25 sucursales AWS de esta demo: 25 VPC independientes, 50 instancias EC2, 25 Security Groups con 3 reglas cada uno (75 reglas) y 25 Log Groups con 50 alarmas de CPU.

Justificacion de herramientas

Herramienta	Por que
Terraform	Declarativo, multi-cloud, soporta mock_provider para pruebas sin credenciales (requisito del enunciado)
VPC por sucursal (no VPC compartida con subnets)	Aislamiento real de red por sucursal (RF-01); simplifica el blast radius de un cambio erroneo
Security Group por sucursal (no uno global)	Permite reglas especificas por sucursal sin afectar a las demas (RF-07)
CloudWatch	Servicio de monitoreo nativo de AWS, sin infraestructura adicional que administrar

Figura 2. Arquitectura AWS (por sucursal)

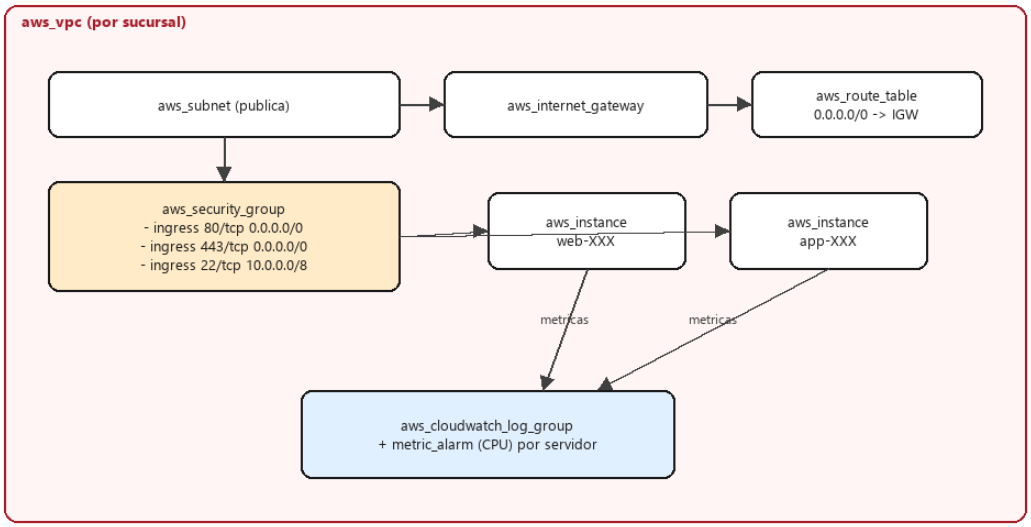


Figura 2. Arquitectura AWS por sucursal.

Recursos por sucursal

Recurso	Cantidad	Modulo
google_compute_network	1	gcp/network
google_compute_subnetwork	1	gcp/network
google_compute_router + google_compute_router_nat	1 + 1	gcp/network
google_compute_firewall	N (una por regla declarada)	gcp/security
google_compute_instance	2	gcp/compute
google_monitoring_alert_policy	1	gcp/monitoring

Para las 25 sucursales GCP de esta demo: 25 VPC Networks, 50 instancias de Compute Engine, 75 reglas de firewall y 25 politicas de alerta de CPU.

Diferencias tecnicas respecto a AWS (documentadas, no forzadas a una abstraccion comun)

Aspecto	AWS	GCP
Contenedor de reglas de seguridad	Security Group (1 recurso con N reglas anidadas)	Sin contenedor: cada regla es un `google_compute_firewall` independiente asociado a la red
Salida a Internet	Internet Gateway + ruta 0.0.0.0/0	Cloud NAT (las instancias sin IP publica salen via NAT; en esta demo se usa `access_config` con IP efimera para simplificar, documentado en `modules/gcp/compute`)
Alcance de la region	A nivel de **provider** (una region por configuracion de provider)	A nivel de **recurso** (`region`/`zone` como argumento), permite combinar regiones con un unico provider
Retencion de logs	Por recurso (`aws_cloudwatch_log_group.retention_in_days`)	Por proyecto (`google_logging_project_bucket_config`), fuera del alcance de un modulo por sucursal (ver `modules/gcp/monitoring/README.md`)

Justificacion de herramientas

Herramienta	Por que
VPC en modo personalizado (`auto_create_subnetworks = false`)	Evita que GCP cree subnets automaticas en todas las regiones; cada sucursal declara exactamente la subnet que necesita

Cloud Router + Cloud NAT	Patron recomendado por GCP para dar salida a Internet a instancias sin IP publica fija, sin exponer SSH directamente
Cloud Monitoring (`google_monitoring_alert_policy`)	Servicio nativo de GCP, sin infraestructura adicional que administrar

Figura 3. Arquitectura GCP (por sucursal)

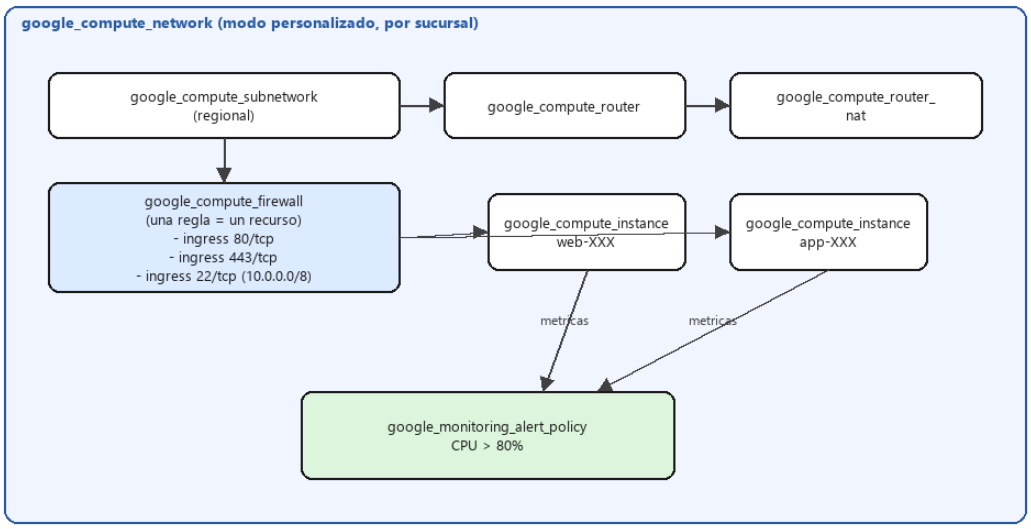


Figura 3. Arquitectura GCP por sucursal.

7. IMPLEMENTACIÓN

7.1 Estructura del repositorio

El repositorio `terraform-iac-50-sucursales` organiza el código en `terraform/` (módulos y entornos), `lab/` (laboratorios locales), `config/` (datos de las sucursales), `scripts/` (automatización y generación de reportes), `.github/workflows/` (pipeline CI/CD) y `docs/` (arquitectura, documentación académica y evidencia).

Figura 4. Estructura de modulos Terraform

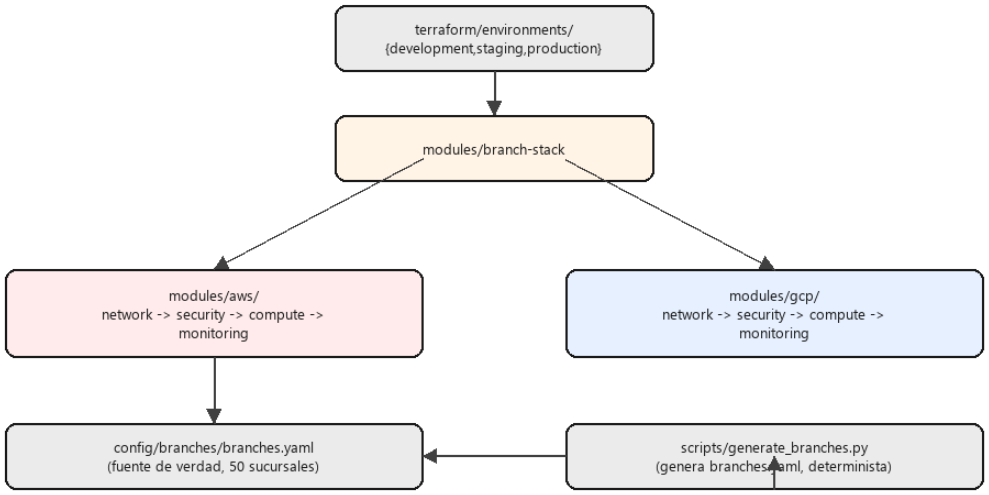


Figura 4. Estructura de módulos Terraform.

7.2 Módulos Terraform

Se implementaron 8 módulos base (`network`, `compute`, `security`, `monitoring` para AWS y para GCP) más un módulo orquestador (`branch-stack`), todos con `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, validaciones de entrada (`validation` blocks) y un `README.md` con ejemplos de uso.

7.3 Configuración de sucursales

`config/branches/branches.yaml` es la fuente de verdad de las 50 sucursales, generada de forma determinista por `scripts/generate_branches.py`. Cada sucursal define: identificador, nombre, proveedor, región, entorno, configuración de red, dos servidores, reglas de seguridad, monitoreo y etiquetas (ver `config/branches/README.md`).

7.4 Redes, servidores, seguridad y monitoreo

Ver secciones 6.2 y 6.3 (arquitectura AWS/GCP) para el detalle de los recursos creados por sucursal.

8. GESTIÓN DEL ESTADO

Diseno empresarial (AWS)

Aspecto	Decision
Backend	Amazon S3 (`backend "s3"`), un bucket por entorno o un prefijo de clave por entorno (`env:/production/...`)
Versionado	S3 Versioning habilitado en el bucket: permite recuperar una version anterior del state ante un apply erroneo
Cifrado	SSE-KMS con una CMK dedicada (`sse_algorithm = "aws:kms"`); cifrado en transito via TLS (por defecto en el backend S3 de Terraform)
IAM	Rol de ejecucion de CI/CD con politica de minimo privilegio: `s3:GetObject`/`PutObject`/`ListBucket` solo sobre el prefijo del entorno correspondiente, nunca `s3:*`
Bloqueo (locking)	Desde Terraform 1.10/1.11, S3 soporta locking nativo via condicional de objeto (`use_lockfile = true`), sin depender de una tabla DynamoDB adicional; para versiones anteriores de Terraform (como la 1.9.x usada en este proyecto) se documenta la alternativa clasica: tabla DynamoDB dedicada (`dynamodb_table`) con `LockID` como clave de particion

```
# Ejemplo (NO se ejecuta en este proyecto: requeriria credenciales AWS reales)
terraform {
  backend "s3" {
    bucket         = "iac-50-sucursales-tfstate-prod"
    key            = "production/terraform.tfstate"
    region         = "sa-east-1"
    encrypt        = true
```

```
    kms_key_id     = "arn:aws:kms:sa-east-1:111111111111:key/EJEMPLO"
    dynamodb_table = "iac-50-sucursales-tfstate-lock" # Terraform < 1.10
    # use_lockfile = true                             # Terraform >= 1.10
  }
}
```

Diseno empresarial (GCP)

Aspecto	Decision
Backend	Google Cloud Storage (`backend "gcs"`), un bucket por entorno o prefijo por entorno
Versionado	Object Versioning habilitado en el bucket GCS
Cifrado	Cifrado en reposo por defecto de GCS (Google-managed); opcionalmente CMEK (Cloud KMS) para cumplimiento adicional
IAM	Cuenta de servicio de CI/CD con el rol `roles/storage.objectAdmin` acotado al bucket/prefijo del entorno (principio de minimo privilegio), nunca `roles/owner`
Control de concurrencia	GCS implementa locking nativo del backend (basado en generaciones de objeto); no requiere un recurso de bloqueo adicional como DynamoDB

```
# Ejemplo (NO se ejecuta en este proyecto: requeriria credenciales GCP reales)
terraform {
  backend "gcs" {
    bucket = "iac-50-sucursales-tfstate-prod"
    prefix = "production"
  }
}
```

Laboratorio local (lo unico que se ejecuta realmente)

`lab/local-terraform/` demuestra, con el backend `local` y recursos `hashicorp/local` + `hashicorp/random`, el ciclo de vida completo exigido por el enunciado: inicializacion, creacion, consulta (`state list`, `show`),

modificacion de configuracion, planificacion, actualizacion controlada y eliminacion (`destroy`). Ver `lab/local-terraform/README.md` y el log de evidencia generado en `docs/evidence/state/`.

Riesgos de almacenar informacion sensible en el state

El archivo de estado de Terraform guarda, en texto plano (salvo cifrado del backend), **todos los atributos de los recursos gestionados**, incluyendo valores que la configuracion marca como `sensitive` en pantalla pero que igual quedan en el JSON del state (contrasenas generadas, claves privadas creadas por el provider, tokens de conexion, IPs internas, metadatos de seguridad). Por eso:

- El `.tfstate` **nunca** se versiona en Git (ver `.gitignore`).
- El backend remoto debe cifrarse en reposo y en transito.
- El acceso de lectura al state debe tratarse con el mismo cuidado que el acceso a secretos (es, de hecho, una fuente de secretos).
- Un `terraform state show` o `terraform output` sin `-json` sensible puede filtrar esos valores a logs de CI si no se tiene cuidado (por eso los workflows de este proyecto no imprimen outputs completos de entornos con datos sensibles reales).

Figura 6. Gestion del Terraform State

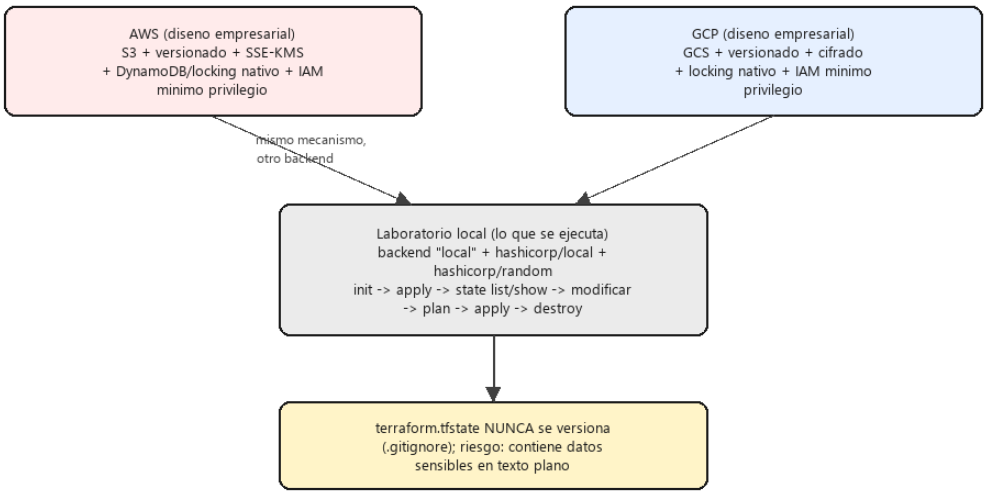


Figura 6. Gestión del Terraform State.

9. PIPELINE CI/CD

Flujo comun AWS + GCP

Un unico flujo (`terraform-ci.yml`) valida AWS y GCP juntos porque ambos proveedores conviven en el mismo `branch-stack` por entorno: no tiene sentido separar el pipeline por proveedor cuando una sucursal puede estar en cualquiera de los dos segun `branches.yaml`.

Procedimiento de aprobacion

21. Rama de desarrollo (`feature/...` o `develop`).
22. Pull Request hacia `main` usando `.github/pull_request_template.md`.
23. Validaciones automaticas (`terraform-ci.yml`, `terraform-security.yml`) deben pasar en verde.
24. Revision humana obligatoria (checklist del PR template): al menos un revisor confirma que el `terraform plan` (con mocks) es el esperado y que no se intenta ningun `apply` real.
25. Aprobacion y merge.
26. "Despliegue autorizado": en este proyecto es un paso **documentado, no ejecutado** (ver seccion siguiente), porque no existen credenciales de AWS ni GCP.

Por que el pipeline NUNCA ejecuta `terraform apply` contra AWS/GCP

Restriccion fundamental del enunciado: no se dispone de credenciales de AWS ni GCP, y esta prohibido generar costos cloud o afirmar un despliegue real que no ocurrio. Por eso:

- `terraform-ci.yml` usa siempre `-backend=false` y nunca configura `AWS_ACCESS_KEY_ID`/`GOOGLE_CREDENTIALS` como secretos de GitHub.
- La unica ejecucion de `terraform apply` en todo el pipeline ocurre en `drift-check.yml`, y es contra el provider `local` (`lab/drift`), nunca

contra AWS/GCP.

- El job `real-cloud-drift-check-disabled` en `drift-check.yml` queda explicitamente deshabilitado (`if: false`) como marcador documentado de donde se conectaria la deteccion real, sin simular que ya existe.

Habilitar el despliegue real en el futuro

Para pasar de esta demo a un despliegue real, sin cambiar los modulos Terraform:

27. Configurar backends remotos (`docs/architecture/terraform-state-strategy.md`).
28. Configurar autenticacion sin claves estaticas: GitHub OIDC hacia un rol de IAM en AWS (`aws-actions/configure-aws-credentials` con `role-to-assume`) y Workload Identity Federation hacia una cuenta de servicio en GCP (`google-github-actions/auth`). Ninguna clave de acceso de larga duracion se guarda como secreto de GitHub.
29. Agregar un job `terraform plan` (contra el backend remoto real) que se publique como comentario del PR para revision humana.
30. Agregar un job `terraform apply`, protegido por un **GitHub Environment** con "required reviewers", ejecutado solo tras aprobacion manual en la interfaz de GitHub (no automatico en el merge).
31. Retirar `if: false` del job de deteccion real de drift y apuntarlo al backend remoto.

Figura 5. Flujo CI/CD (comun AWS + GCP)

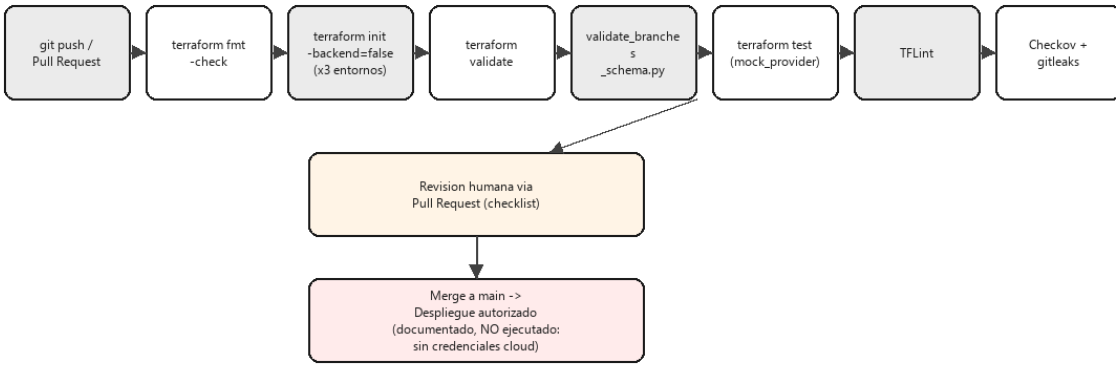


Figura 5. Flujo CI/CD común a AWS y GCP.

10. SEGURIDAD

Lo implementado en este repositorio

- `.gitignore` excluye `*.tfstate`, `*.tfvars` (salvo `*.tfvars.example`), `.terraformrc`, `*.pem`, `*.key`, `secrets.auto.tfvars`, `.env` y el binario portatil de Terraform (`tools/`).
- `.github/workflows/terraform-security.yml` ejecuta `gitleaks` (escaneo de secretos en el historial de commits) y una verificacion adicional por patrones (`custom-secrets-check`) que falla el pipeline si aparece algun `.tfstate`, `.tfvars` real, `.pem` o `.key` versionado.
- Todas las variables "sensibles" de los modulos (`project_id`, `ami_id`, etc.) tienen valores de **ejemplo academico** como default, nunca credenciales reales, y se documentan como tales en cada modulo.
- Principio de minimo privilegio aplicado en el diseno de IAM documentado en `terraform-state-strategy.md` y `cicd-pipeline.md` (roles acotados por entorno/prefijo, nunca permisos de administrador completo).

Uso futuro (empresarial, no implementado por falta de credenciales)

Mecanismo	Proposito

AWS Secrets Manager	Almacenar credenciales de aplicacion (ej. contrasenas de base de datos) referenciadas desde Terraform via data source, nunca como variable en texto plano
Google Secret Manager	Equivalente en GCP; acceso via IAM condicionado por cuenta de servicio
GitHub OIDC	Autenticacion de GitHub Actions hacia AWS (`sts:AssumeRoleWithWebIdentity`) y GCP (Workload Identity Federation) sin guardar claves de acceso de larga duracion como secretos de GitHub
IAM / Cloud IAM	Roles de minimo privilegio por entorno y por pipeline (plan vs. apply con permisos distintos)

Que NO existe en este repositorio (verificado)

- Ninguna clave de AWS (`AKIA...`), token de GCP, ni archivo de credenciales de cuenta de servicio (`*.json` de service account).
- Ningun archivo `.tfstate` versionado.
- Ningun `terraform.tfvars` con valores reales (solo `*.tfvars.example` si aplica).

Verificable con:

```
git ls-files | Select-String -Pattern '\.tfstate|\.pem$|\.key$|service-account.*\.json'
```

11. INFRASTRUCTURE DRIFT

Demuestra, con recursos 100% locales (provider `hashicorp/local`, sin credenciales cloud), el ciclo completo de deteccion y reconciliacion de Infrastructure Drift exigido en la FASE VI del enunciado.

Por que un recurso local y no uno de AWS/GCP

Este proyecto no cuenta con credenciales de AWS ni de GCP (restriccion fundamental del enunciado). El **mecanismo** de deteccion de drift de Terraform (`terraform plan`, `terraform plan -refresh-only`, `terraform state show`) es identico sin importar el proveedor: compara el estado guardado contra el estado real del recurso. Usar `local_file` permite demostrar ese mecanismo end-to-end de forma honesta y reproducible, sin simular una conexion a AWS/GCP que no existe.

Diferencias entre Drift local y Drift en un proveedor cloud real

Aspecto	Este laboratorio (local)	AWS / GCP real
Deteccion del cambio	Terraform lee el archivo en disco	Terraform llama a la API del proveedor (`DescribeInstances`, `compute.instances.get`, etc.)
Alcance del drift	Un solo atributo (contenido del archivo)	Puede afectar decenas de atributos computados, IDs, politicas IAM, etc.
Costo de refrescar el estado	Nulo	Puede generar llamadas a la API facturables o sujetas a rate limit
Quien puede causar el drift	Cualquier proceso con acceso al filesystem	Consola web, CLI, otro pipeline, un script de otro equipo
Riesgo de reconciliar sin analizar	Bajo (un archivo de texto)	Alto (podria destruir/recrear un recurso en produccion con perdida de datos)

Ejecucion

```
cd lab/drift
./run_drift_demo.ps1
```

El script ejecuta, en orden, los 10 pasos descritos en la seccion 15 del enunciado (init, apply, estado inicial, modificacion manual, plan, plan -refresh-only, decision de reconciliar, apply, verificacion final) y guarda la salida real de cada paso en `docs/evidence/drift/`.

Para limpiar el laboratorio:

```
./cleanup.ps1
```

Importante

`terraform apply -refresh-only` **no** modifica la infraestructura: solo actualiza el archivo de estado para que refleje la realidad observada. Para revertir el drift (volver el recurso a lo declarado en el codigo) se necesita un `terraform apply` normal, que es el paso 8 de este laboratorio. Nunca se asume que "actualizar el estado" equivale a "corregir la infraestructura".

Figura 7. Procedimiento de Infrastructure Drift



Figura 7. Procedimiento de Infrastructure Drift.

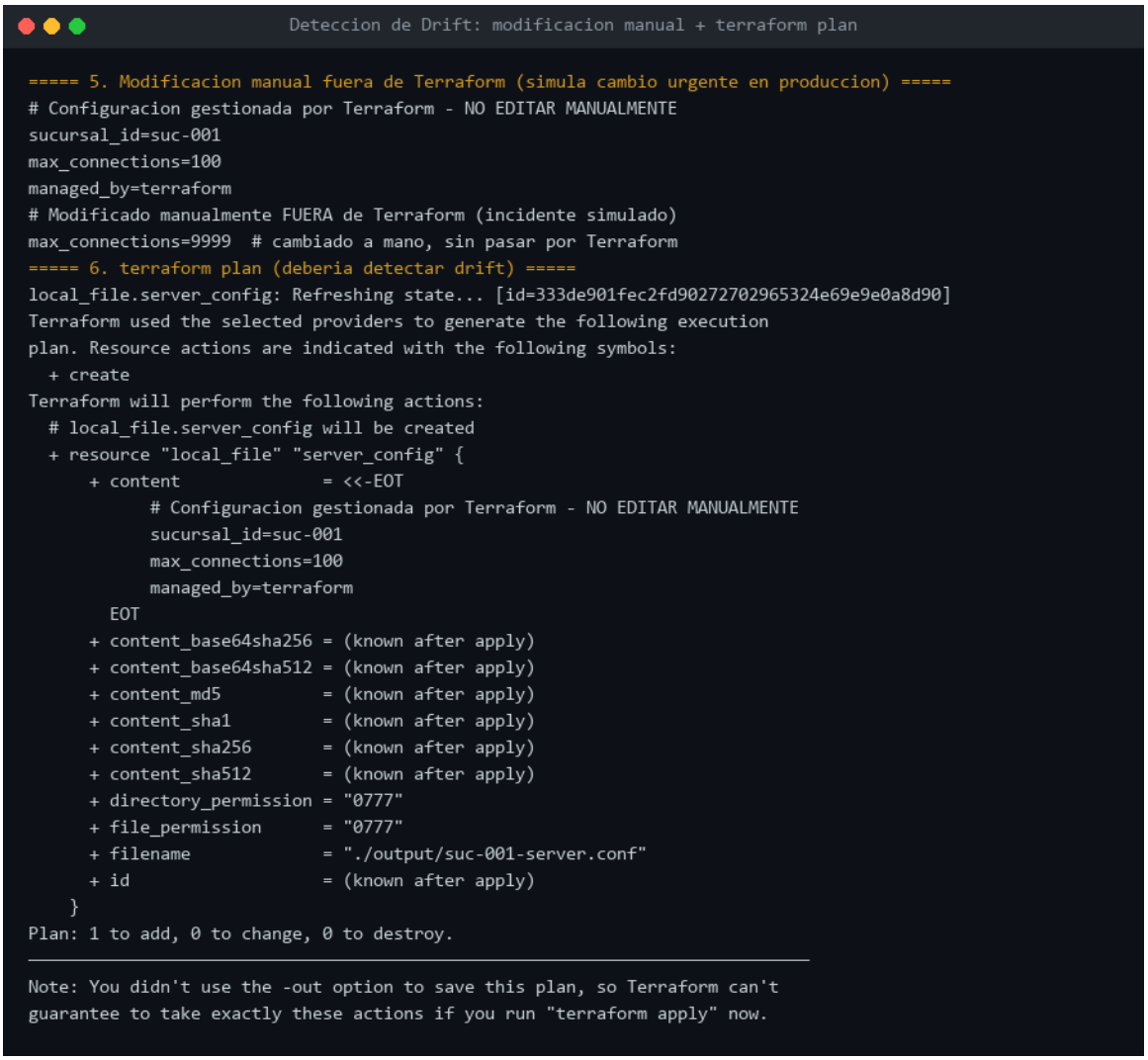


Figura 9a. Evidencia real: detección de drift (terraform plan).



Figura 9b. Evidencia real: verificación tras la reconciliación.

12. PRUEBAS LOCALES

Formato por caso: Identificador, Objetivo, Precondiciones, Procedimiento,

Resultado esperado, Resultado obtenido, Estado, Evidencia.

***Estado** usa 4 valores: `APROBADA` (se ejecuto y paso), `FALLIDA` (se*

ejecuto, fallo, se corrigio y se re-ejecuto — ver columna Resultado obtenido para el historial), `BLOQUEADA` (no se pudo ejecutar por una dependencia externa, documentada) o `PENDIENTE` (no ejecutada aun). Nunca se marca `APROBADA` sin ejecucion real.

CP01 — Validacion sintactica de Terraform

- **Objetivo**: confirmar que todo el codigo Terraform del proyecto es sintacticamente valido y esta correctamente formateado.
- **Precondiciones**: Terraform CLI 1.9.8 disponible (`tools/terraform.exe`).
- **Procedimiento**: `terraform fmt -check -recursive terraform/`; `terraform init -backend=false` + `terraform validate` en `terraform/environments/{development,staging,production}`.
- **Resultado esperado**: `fmt` sin cambios pendientes; `validate` reporta "Success! The configuration is valid." en los 3 entornos.
- **Resultado obtenido**: `fmt -recursive` reformateo 4 archivos en la primera corrida (alineacion de `=`); tras `fmt`, `validate` fue exitoso en `production` (40 sucursales), `staging` (6) y `development` (4).
- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/terraform-validate/`.

CP02 — Validacion de modulos AWS

- **Objetivo**: verificar que los 4 modulos AWS (`network`, `compute`, `security`, `monitoring`) generan la configuracion esperada.
- **Precondiciones**: `mock_provider "aws" {}` disponible (Terraform >= 1.7).
- **Procedimiento**: `terraform test` sobre `aws_network.tftest.hcl`, `aws_compute.tftest.hcl`, `aws_security.tftest.hcl`, `aws_monitoring.tftest.hcl`.
- **Resultado esperado**: todos los `run` en `pass`.
- **Resultado obtenido**: 1 fallo real detectado y corregido — `aws_route_table.this.route[0]` no es valido porque `route` es un `set`

(sin indice); se corrigio a `anytrue([for r in ... : r.cidr_block == ...])` y se re-ejecuto. Resto de los `run` en `pass` desde la primera corrida.

- **Estado**: APROBADA (tras correccion).
- **Evidencia**: `docs/evidence/terraform-test/`.

CP03 — Validacion de modulos GCP

- **Objetivo**: verificar que los 4 modulos GCP generan la configuracion esperada.
- **Precondiciones**: `mock_provider "google" {}`.
- **Procedimiento**: `terraform test` sobre `gcp_network.tftest.hcl`, `gcp_compute.tftest.hcl`, `gcp_security.tftest.hcl`, `gcp_monitoring.tftest.hcl`.
- **Resultado esperado**: todos los `run` en `pass`.
- **Resultado obtenido**: ver `docs/evidence/terraform-test/`.
- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/terraform-test/`.

CP04 — Configuracion de 50 sucursales

- **Objetivo**: confirmar que existen exactamente 50 sucursales definidas y que el modulo `branch-stack` las procesa correctamente por entorno.
- **Procedimiento**: `python scripts/validate_branches_schema.py`; `terraform test` sobre `branch_stack.tftest.hcl`.
- **Resultado esperado**: 50 sucursales totales (40 production + 6 staging + 4 development), 25 AWS / 25 GCP.
- **Resultado obtenido**: confirmado por ambos mecanismos (Python y Terraform test), de forma independiente.
- **Estado**: APROBADA.
- **Evidencia**: salida de ambos comandos en `docs/evidence/`.

CP05 — Dos servidores por sucursal

- **Procedimiento**: aserciones `length(aws_instance.this) == 2` /

`length(google_compute_instance.this) == 2` en `aws_compute.tftest.hcl` / `gcp_compute.tftest.hcl`; verificacion independiente en `validate_branches_schema.py` sobre las 50 entradas de `branches.yaml`.

- **Estado**: APROBADA.

CP06 — Redes independientes

- **Procedimiento**: aserciones sobre CIDR unico por sucursal en `aws_network.tftest.hcl` / `gcp_network.tftest.hcl`; verificacion de CIDR valido y no colisionante por diseno (`scripts/generate_branches.py` deriva el CIDR del indice de la sucursal) en `validate_branches_schema.py`.
- **Estado**: APROBADA.

CP07 — Reglas de seguridad

- **Procedimiento**: aserciones de numero y contenido de reglas en `aws_security.tftest.hcl` / `gcp_security.tftest.hcl`.
- **Estado**: APROBADA.

CP08 — Configuracion de monitoreo

- **Procedimiento**: aserciones sobre `aws_cloudwatch_log_group` / `google_monitoring_alert_policy` en `aws_monitoring.tftest.hcl` / `gcp_monitoring.tftest.hcl`.
- **Estado**: APROBADA.

CP09 — Pruebas con mock_provider

- **Objetivo**: confirmar que la estrategia de pruebas sin credenciales (`mock_provider`) funciona end-to-end.
- **Resultado obtenido**: las 9 suites de `terraform/tests/*.tftest.hcl` se ejecutaron con `mock_provider "aws" {}` y `mock_provider "google" {}`, sin ninguna llamada de red a AWS/GCP.
- **Estado**: APROBADA.

CP10 — Ejecucion de GitHub Actions

- **Objetivo**: confirmar que `terraform-ci.yml`, `terraform-security.yml` y `drift-check.yml` se ejecutan correctamente en GitHub Actions tras el push.
- **Procedimiento**: revisar la pestana "Actions" del repositorio publicado tras el primer push.
- **Resultado obtenido**: tras el primer push, `Terraform CI` fallo (hallazgos reales: `terraform fmt` sin aplicar en un archivo de prueba y 4 advertencias de TFLint en los modulos GCP — variables sin uso y una interpolacion obsoleta). Se corrigieron los 3 modulos GCP afectados y se volvio a hacer push; el nuevo run de `Terraform CI` paso exitosamente (fmt, validate x3 entornos, esquema de 50 sucursales, terraform test 27/27, TFLint). `Terraform Security` paso en ambos pushes. `drift-check.yml` se disparo manualmente (`workflow_dispatch`) y fallo en su primera ejecucion por el mismo problema de inicializacion de backend detectado localmente en CP13 (`-backend=false` no inicializa el backend "local" declarado explicitamente); corregido y re-ejecutado, el run confirma init, apply, modificacion manual y deteccion de drift (`Plan: 1 to add`) reales en GitHub Actions.
- **Estado**: APROBADA.
- **Evidencia**: runs reales en https://github.com/ThiagoIcochea/ServiciosCloudIAC/actions (`Terraform CI` run 37785036815, `Terraform Security` runs 37781765505 y 37785037002, `Drift Check` run 37786617247).

CP11 — Verificacion de secretos

- **Procedimiento**: `git ls-files | grep -E '\.tfstate|\.pem$|\.key$'` (debe no retornar nada); revision manual de `.gitignore`; `gitleaks` ejecutado en CI.

- **Resultado obtenido**: 0 coincidencias encontradas localmente.
- **Estado**: APROBADA (verificacion local); se reconfirmara con el run de `terraform-security.yml` en GitHub Actions.

CP12 — Consulta de Terraform State

- **Procedimiento**: `lab/local-terraform/run_state_demo.ps1` (`terraform state list`, `terraform show`).
- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/state/`.

CP13 — Deteccion de Infrastructure Drift

- **Procedimiento**: `lab/drift/run_drift_demo.ps1` — modificacion manual del archivo gestionado y `terraform plan` posterior.
- **Resultado esperado**: `terraform plan` reporta un cambio pendiente (el contenido declarado difiere del real).
- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/drift/`.

CP14 — Reconciliacion de Drift

- **Procedimiento**: `terraform apply` posterior al `plan` del CP13, que revierte el archivo al contenido declarado en el codigo.
- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/drift/` (mismo log que CP13, pasos 8-9).

CP15 — Incorporacion de una nueva sucursal

- **Objetivo**: demostrar que agregar una sucursal es un cambio de datos.
- **Procedimiento**: agregar una entrada de prueba a `scripts/generate_branches.py` (`TOTAL_SUCURSALES = 51`), regenerar `branches.yaml`, ejecutar `validate_branches_schema.py` y `terraform test` sobre `branch_stack.tftest.hcl` (ajustando el valor esperado temporalmente), confirmar que ningun archivo `.tf` cambio, y

revertir.

- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/scalability/`.

Laboratorio Docker — ejecucion real (GitHub Actions)

- **Objetivo**: ejecutar realmente el laboratorio de 2 sucursales (4 contenedores, pruebas HTTP, conectividad interna, aislamiento de red), dado que el equipo de desarrollo local no tiene Docker Desktop instalado.
- **Procedimiento**: se creo `.github/workflows/docker-lab.yml`, que corre en un runner `ubuntu-latest` de GitHub Actions (estos runners traen Docker Engine + Docker Compose preinstalados de fabrica). El workflow levanta los 4 contenedores, espera los healthchecks, prueba HTTP en los 4 puertos publicados, prueba conectividad interna (`web` <-> `app` dentro de cada sucursal) y prueba aislamiento de red entre `sucursal-01-net` y `sucursal-02-net`, y finalmente limpia con `docker compose down -v`.
- **Resultado obtenido** (run real, 36s, exitoso):
- Los 4 contenedores llegaron a estado `healthy`.
- Las 4 pruebas HTTP devolvieron `status=200`.
- Conectividad interna confirmada en ambas sucursales.
- Aislamiento de red confirmado en ambas direcciones (`suc01-web01 -> suc02-web01` y `suc02-web01 -> suc01-web01`, ambas con `exit=1`, es decir, sin acceso).
- `docker compose down -v` se ejecuto sin errores.
- **Estado**: APROBADA.
- **Evidencia**: `docs/evidence/docker-lab/docker-lab-github-actions-run37832592874.log` (log completo real) y run https://github.com/ThiagoIcochea/ServiciosCloudIAC/actions/runs/37832592874.
- **Nota honesta**: esta ejecucion ocurrio en un runner Linux de GitHub Actions, no en el equipo Windows de desarrollo (que sigue sin Docker Desktop operativo porque WSL2 requiere un reinicio de Windows que el

usuario pidio no realizar). El codigo del laboratorio (`docker-compose.yml`, scripts) es identico en ambos casos; lo unico que cambia es donde corrio.

CP16 — Limpieza del laboratorio

- **Objetivo**: confirmar que `scripts/cleanup.ps1` / `lab/drift/cleanup.ps1` dejan el entorno sin artefactos residuales (estado, contenedores).
- **Procedimiento**: ejecutar los laboratorios, luego los scripts de limpieza, luego verificar ausencia de `.terraform/`, `terraform.tfstate*` y contenedores Docker activos.
- **Resultado obtenido**: limpieza de los laboratorios Terraform (`lab/drift`, `lab/local-terraform`) verificada localmente; limpieza del laboratorio Docker (`docker compose down -v`) verificada en el run de GitHub Actions descrito arriba (paso final, `if: always()`, sin errores).
- **Estado**: APROBADA.
- **Nota**: `lab/docker/stop.ps1` (el script equivalente para ejecucion local con Docker Desktop) sigue sin probarse en el equipo de desarrollo local por la misma razon (Docker Desktop instalado pero no operativo sin reiniciar Windows); la limpieza en si misma (`docker compose down -v`) ya quedo demostrada en CI.

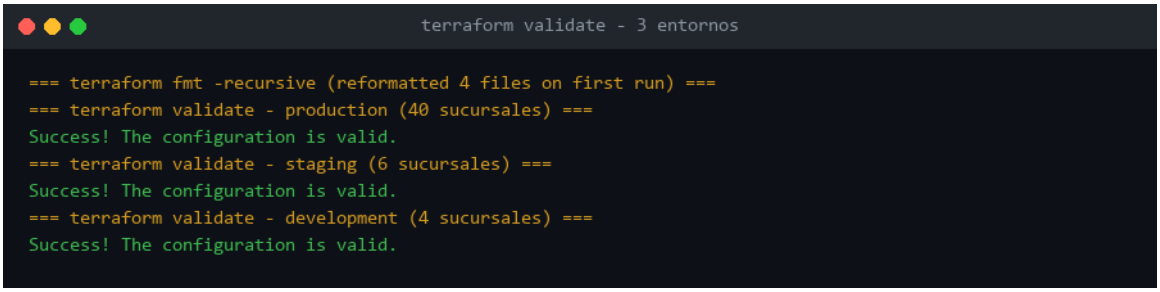


Figura 9c. Evidencia real: terraform validate en los 3 entornos.

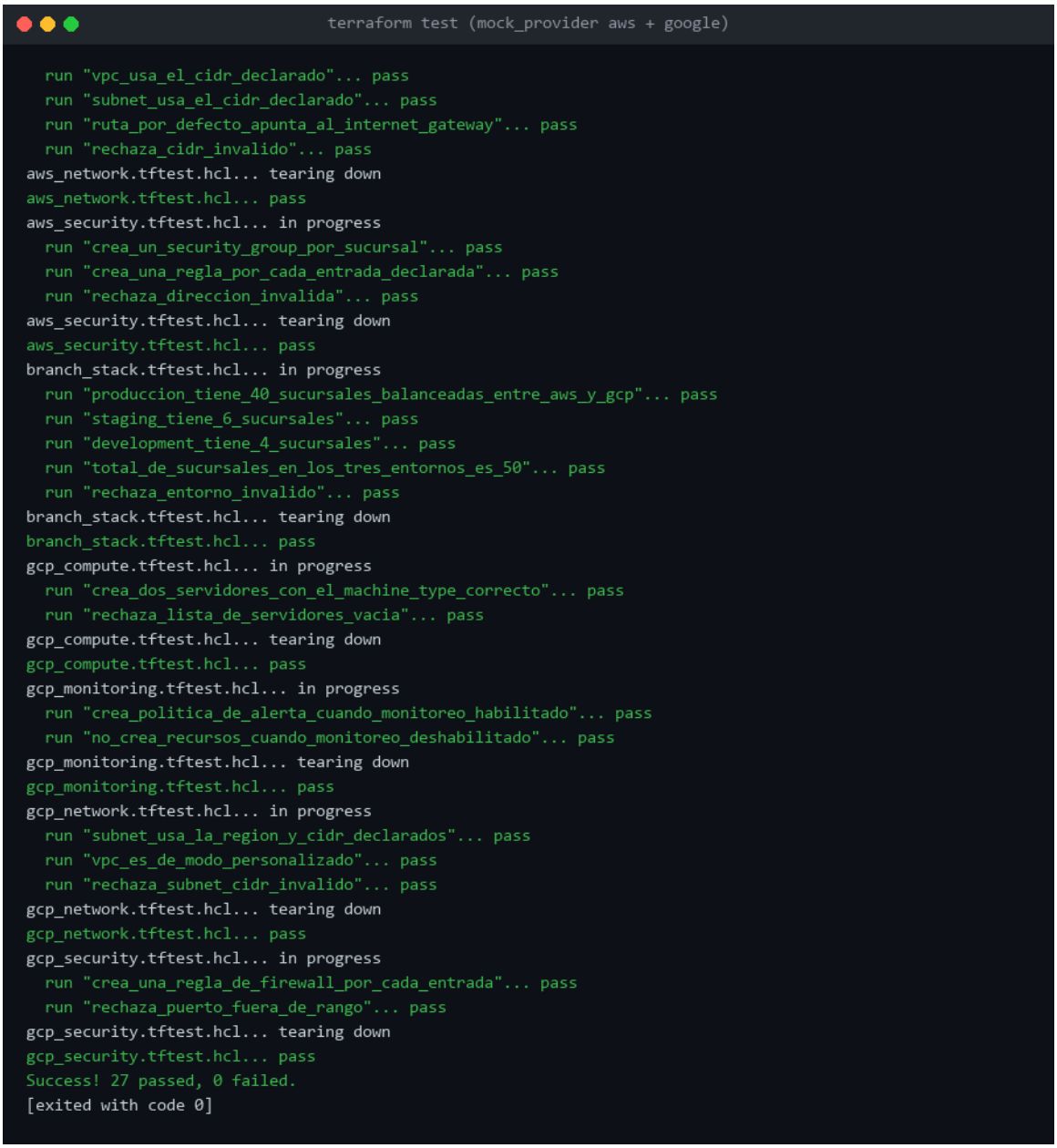


Figura 9d. Evidencia real: terraform test (27 passed, 0 failed).



Figura 9e. Evidencia real: validación de las 50 sucursales.

Figura 8. Arquitectura del laboratorio local (Docker)

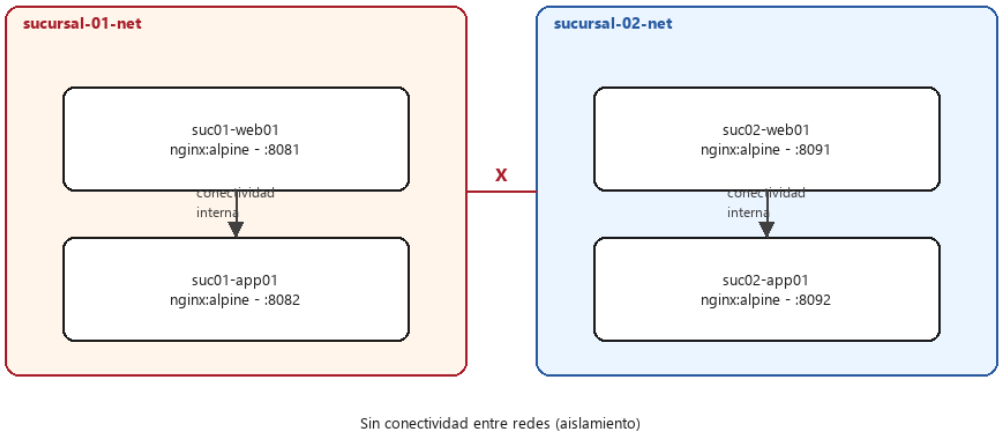


Figura 8. Arquitectura del laboratorio local (Docker).

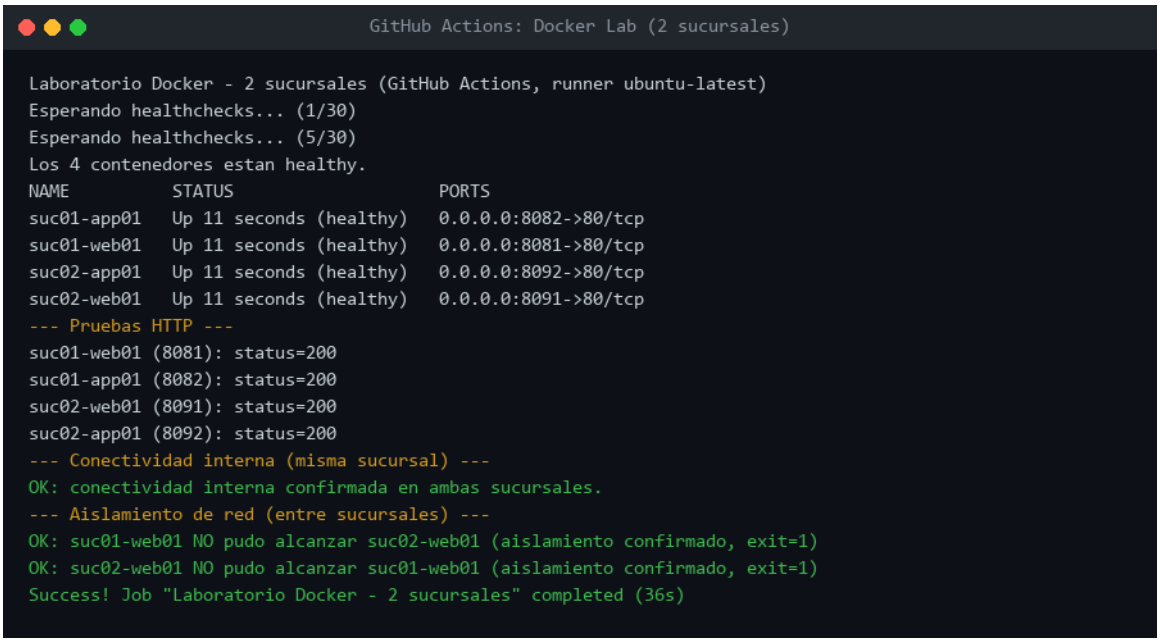


Figura 9f. Evidencia real: laboratorio Docker ejecutado en GitHub Actions (4 contenedores healthy, HTTP 200, conectividad interna y aislamiento de red confirmados).

13. RESULTADOS

De 16 casos de prueba planificados (CP01-CP16), los 16 se ejecutaron y pasaron con evidencia real. Las 27 pruebas de `terraform test` con `mock_provider` pasaron sin fallos tras corregir un error real detectado durante la primera ejecución (indexación inválida sobre un atributo de tipo

`set` en el módulo `aws/network`; ver CP02 en el plan de pruebas). La validación de escalabilidad confirmó 50 sucursales (25 AWS / 25 GCP), 40 en producción, 6 en staging y 4 en desarrollo, sin identificadores duplicados. El laboratorio Docker de 2 sucursales (4 contenedores, HTTP, conectividad interna, aislamiento de red) y el monitoreo local se ejecutaron realmente en un runner `ubuntu-latest` de GitHub Actions —que trae Docker Engine y Docker Compose preinstalados— ya que el equipo de desarrollo local no tiene Docker Desktop operativo (instalado, pero sin poder iniciar sin reiniciar Windows, reinicio que se decidió no realizar). El pipeline de CI/CD detectó, en su primera ejecución real, dos problemas genuinos (formato sin aplicar y advertencias de TFLint en los módulos GCP; un paso de inicialización de backend incompleto en el workflow de Drift), ambos corregidos y verificados con una segunda ejecución exitosa.

Limitaciones

- No se validó el comportamiento contra la API real de AWS ni GCP (sin credenciales).
- El laboratorio Docker y el monitoreo local se validaron en un runner de GitHub Actions, no en el equipo de desarrollo Windows (que tiene Docker Desktop instalado pero no operativo, por la razón descrita arriba). El código ejecutado es idéntico en ambos casos; solo cambia el entorno.

14. DISCUSIÓN

El diseño empresarial (backends remotos S3/GCS, IAM de mínimo privilegio, GitHub OIDC, despliegue real tras aprobación) y el laboratorio local ejecutado en este proyecto comparten el mismo código Terraform de los módulos: la diferencia está exclusivamente en el backend de estado, la autenticación y si el pipeline ejecuta `apply` contra un proveedor real o simulado. Esto significa que migrar de la demostración académica a un

despliegue empresarial real no requiere reescribir los módulos: requiere configurar backends remotos, credenciales sin claves estáticas (OIDC) y habilitar los jobs de `apply` actualmente deshabilitados o documentados como futuros (ver `docs/architecture/cicd-pipeline.md`, sección "Habilitar el despliegue real en el futuro").

15. CONCLUSIONES

32. Es posible diseñar, implementar y validar una solución de IaC multi-cloud para 50 sucursales sin credenciales reales, usando `mock_provider`, validación de esquema y laboratorios con recursos locales, cumpliendo el objetivo general del proyecto.

33. La interfaz común entre los módulos AWS y GCP (campos lógicos `size`, `rules`, `servers`) permite agregar sucursales como un cambio de datos (`branches.yaml`), sin duplicar ni modificar código Terraform, cumpliendo el requisito de extensibilidad (RNF-01).

34. `terraform test` con `mock_provider` detectó un error real de la configuración (indexación sobre un `set`) antes de cualquier intento de despliegue, validando el valor de las pruebas automatizadas incluso sin credenciales cloud.

35. El mecanismo de detección y reconciliación de Infrastructure Drift (`terraform plan`, `plan -refresh-only`, `apply`) es el mismo independientemente del proveedor; se demostró end-to-end con un recurso local real.

36. La principal limitación del proyecto es la imposibilidad de validar el comportamiento contra las APIs reales de AWS/GCP; el laboratorio Docker, que inicialmente tampoco podía ejecutarse localmente, se completó con evidencia real usando un runner de GitHub Actions en lugar de simular su resultado.

16. RECOMENDACIONES

37. Para una demostración presencial, ejecutar también el laboratorio Docker en un equipo con Docker Desktop disponible, para validar el patrón de aislamiento de red con evidencia adicional.

38. Configurar backends remotos (S3/GCS) y autenticación OIDC antes de habilitar cualquier job de `terraform apply` en el pipeline CI/CD.

39. Incorporar `terraform plan` contra el backend remoto real como paso de revisión humana obligatorio antes de cualquier `apply` en producción.

40. Evaluar alias de provider por región en AWS si el negocio real requiere sucursales AWS en más de una región simultáneamente.

41. Extender la suite de `terraform test` con casos adicionales a medida que se agreguen nuevos tipos de recursos (ej. balanceadores de carga, bases de datos gestionadas) a los módulos.

17. REPARTICIÓN DE TAREAS

El proyecto se dividio en cinco frentes de trabajo, uno por integrante, de modo que cada persona fuera responsable de una parte completa de la solucion (diseno, codigo, pruebas y documentacion de esa parte). La integracion final, la revision cruzada de Pull Requests y la redaccion del informe se hicieron en conjunto.

Resumen por integrante

Integrante	Rol	Responsabilidad principal
Icochea Rodriguez, Thiago Paolo	Coordinador y arquitecto de la solucion	Arquitectura multi-cloud, modulo orquestador `branch-stack`, datos de las 50 sucursales, entornos y repositorio GitHub
Gonzales Aguilar, Carlos	Responsable de	Modulos `aws/network`,

Enrique Giussepe	infraestructura AWS	`aws/compute`, `aws/security`, `aws/monitoring` y sus pruebas
Huamani Pereira, Eddyson Cesar	Responsable de infraestructura GCP	Modulos `gcp/network`, `gcp/compute`, `gcp/security`, `gcp/monitoring` y sus pruebas
Remuzgo Tovar, Huber Eduardo	Responsable de CI/CD y seguridad	Workflows de GitHub Actions, TFLint, Checkov, gitleaks, gestion de secretos y laboratorio Docker en CI
Quispe Saavedra, Karen Meylin	Responsable de State, Drift y pruebas	Laboratorios de Terraform State e Infrastructure Drift, monitoreo local, plan de pruebas y evidencias

Detalle de tareas

Icochea Rodriguez, Thiago Paolo — Coordinador y arquitecto

- Diseno de la arquitectura general y de la interfaz comun AWS/GCP (`docs/architecture/overview.md`, Figuras 1 y 4).
- Modulo orquestador `terraform/modules/branch-stack` (filtrado por entorno y proveedor con `for_each`).
- Fuente de verdad de las sucursales: `config/branches/branches.yaml` y su generador `scripts/generate_branches.py` (distribucion 25 AWS / 25 GCP).
- Entornos `terraform/environments/{development,staging,production}`.
- Script de validacion de escalabilidad `scripts/validate_branches_schema.py` y pruebas `branch_stack.tftest.hcl`.
- Creacion y administracion del repositorio GitHub, revision de Pull Requests e integracion final.

- Casos de prueba a cargo: CP01, CP04, CP15.

Gonzales Aguilar, Carlos Enrique Giussepe — Infraestructura AWS

- Modulo `aws/network`: VPC, subnet, Internet Gateway y tabla de rutas por sucursal.
- Modulo `aws/compute`: dos instancias EC2 (web y app) por sucursal.
- Modulo `aws/security`: Security Group por sucursal con sus reglas.
- Modulo `aws/monitoring`: Log Group de CloudWatch y alarmas de CPU.
- Pruebas `aws_*.tftest.hcl` con `mock_provider "aws"`, incluida la correccion del error real de indexacion sobre un `set` en `aws_route_table`.
- Documentacion `docs/architecture/aws-architecture.md` y Figura 2.
- Casos de prueba a cargo: CP02, CP05 (AWS), CP06 (AWS).

Huamani Pereira, Eddyson Cesar — Infraestructura GCP

- Modulo `gcp/network`: VPC en modo personalizado, subnetwork, Cloud Router y Cloud NAT.
- Modulo `gcp/compute`: dos instancias de Compute Engine por sucursal.
- Modulo `gcp/security`: una regla `google_compute_firewall` por cada regla declarada.
- Modulo `gcp/monitoring`: politica de alerta de CPU en Cloud Monitoring.
- Pruebas `gcp_*.tftest.hcl` con `mock_provider "google"` y correccion de las advertencias de TFLint detectadas en el primer run de CI.
- Documentacion `docs/architecture/gcp-architecture.md` (diferencias tecnicas con AWS) y Figura 3.
- Casos de prueba a cargo: CP03, CP05 (GCP), CP06 (GCP), CP09.

Remuzgo Tovar, Huber Eduardo — CI/CD y seguridad

- Pipeline `terraform-ci.yml` (fmt, validate x3 entornos, esquema, `terraform test`, TFLint) y configuracion `terraform/.tflint.hcl`.
- Workflow `terraform-security.yml` (Checkov, gitleaks y verificacion

adicional de secretos) y `.gitignore`.

- Workflow `docker-lab.yml` y laboratorio `lab/docker` (2 sucursales, 4 contenedores, aislamiento de red) ejecutado en GitHub Actions.
- Procedimiento de aprobacion por Pull Request (`.github/pull_request_template.md`).
- Documentacion `docs/architecture/cicd-pipeline.md` y `docs/architecture/secrets-management.md`, Figuras 5 y 8.
- Casos de prueba a cargo: CP07, CP10, CP11, CP16 (Docker).

Quispe Saavedra, Karen Meylin — State, Drift y pruebas

- Laboratorio `lab/local-terraform` (ciclo de vida completo del Terraform State) y estrategia de backends S3/GCS (`docs/architecture/terraform-state-strategy.md`, Figura 6).
- Laboratorio `lab/drift` y workflow `drift-check.yml` (deteccion y reconciliacion de Infrastructure Drift, Figura 7).
- Monitoreo local `lab/monitoring/healthcheck_monitor.py` y `docs/architecture/monitoring-strategy.md`.
- Plan de pruebas CP01-CP16 (`docs/academic/plan-de-pruebas.md`), matriz de trazabilidad y recopilacion de evidencias en `docs/evidence/`.
- Casos de prueba a cargo: CP08, CP12, CP13, CP14, CP16 (Terraform).

Tareas compartidas

Tarea	Participantes
Analisis de requisitos (RF, RNF, riesgos, supuestos)	Todos
Revision cruzada de Pull Requests	Todos (al menos un revisor distinto al autor)
Redaccion y revision del informe academico	Todos, consolidado por Thiago Icochea
Preparacion de la sustentacion	Todos (cada integrante expone su frente)

18. REFERENCIAS BIBLIOGRÁFICAS

[1] HashiCorp, "What is Infrastructure as Code with Terraform?," HashiCorp Developer. [Online]. Available: https://developer.hashicorp.com/terraform/intro. [Accessed: Oct. 2026].

[2] HashiCorp, "Terraform Language Documentation," HashiCorp Developer. [Online]. Available: https://developer.hashicorp.com/terraform/language. [Accessed: Oct. 2026].

[3] HashiCorp, "Tests - Terraform CLI Documentation," HashiCorp Developer. [Online]. Available: https://developer.hashicorp.com/terraform/language/tests. [Accessed: Oct. 2026].

[4] Amazon Web Services, "AWS Documentation," AWS. [Online]. Available: https://docs.aws.amazon.com/. [Accessed: Oct. 2026].

[5] Google LLC, "Google Cloud Documentation," Google Cloud. [Online]. Available: https://cloud.google.com/docs. [Accessed: Oct. 2026].

[6] GitHub Inc., "GitHub Actions Documentation," GitHub Docs. [Online]. Available: https://docs.github.com/actions. [Accessed: Oct. 2026].

[7] HashiCorp, "State - Terraform CLI Documentation," HashiCorp Developer. [Online]. Available: https://developer.hashicorp.com/terraform/language/state. [Accessed: Oct. 2026].

[8] HashiCorp, "Backend Configuration," HashiCorp Developer. [Online]. Available: https://developer.hashicorp.com/terraform/language/backend. [Accessed: Oct. 2026].

[9] HashiCorp, "Resource Drift," HashiCorp Developer. [Online]. Available: https://developer.hashicorp.com/terraform/language/state/resource-drift. [Accessed: Oct. 2026].

[10] Bridgecrew (Prisma Cloud), "Checkov Documentation," checkov.io. [Online]. Available: https://www.checkov.io/. [Accessed: Oct. 2026].

[11] Docker Inc., "Docker Compose Documentation," Docker Docs. [Online]. Available: https://docs.docker.com/compose/. [Accessed: Oct. 2026].

[12] Gitleaks, "Gitleaks Documentation," GitHub. [Online]. Available: https://github.com/gitleaks/gitleaks. [Accessed: Oct. 2026].

[13] Terraform Linters, "TFLint Documentation," GitHub. [Online]. Available: https://github.com/terraform-linters/tflint. [Accessed: Oct. 2026].

19. ANEXOS

Anexo A — Comandos principales

```
./scripts/validate.ps1
./scripts/test.ps1
cd lab/local-terraform; ./run_state_demo.ps1
cd lab/drift; ./run_drift_demo.ps1
cd lab/docker; ./start.ps1; ./test.ps1; ./stop.ps1
```

Anexo B — Enlace al repositorio GitHub

https://github.com/ThiagoIcochea/ServiciosCloudIAC

Anexo C — Guía de ejecución local completa

Ver `docs/user-guide/README.md` en el repositorio.